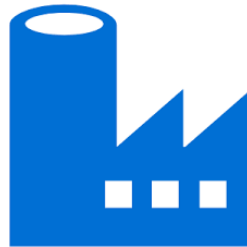




Web: inceptez.com Mail: info@inceptez.com Call: 7871299810, 7871299817

Azure Data Factory



ADF Components – Triggers

Triggers are used to automatically start pipelines based on time or events.

Instead of running a pipeline manually, triggers help automate execution.

Types of Triggers in ADF

ADF supports **three main types of triggers**:

1. Schedule Trigger

Runs pipelines on a **time-based schedule**.

Used for:

- Hourly jobs
- Daily loads
- Weekly or monthly runs

Ex: *Run pipeline every day at 1 AM*

2. Tumbling Window Trigger

Runs pipelines in

fixed, non-overlapping time windows.

Used for:

- Incremental data loads
- Time-sliced processing
- Exactly-once execution

Key Features:

- Each window runs once
- Supports retry and backfill
- Maintains state

Ex: *Run every 15 minutes*

Process data for that 15-minute window

3. Event-Based Trigger

Runs pipelines when a **specific event occurs**.

Used for:

- File arrival
- Blob creation or deletion
- Event-driven ingestion

Ex:

*When a file is uploaded to Blob Storage →
Run pipeline*

ADF Components – Data Flow

- A **Data Flow** is to **clean, join, filter, enrich, and aggregate data** using a visual designer.
- Under the hood, ADF automatically runs these transformations on **Apache Spark**.
- Design visually → ADF executes it on Spark → no cluster management needed.
- Data Flow components are the **building blocks** used to design transformations.

They fall into 3 categories:

◆ Source

- Reads data into the Data Flow
- Connects to datasets (e.g., ADLS, Blob, SQL DB, Delta)
- Defines:
 - File format (CSV/Parquet)
 - Schema
 - Projection
 - Sampling

Use: Bring data from storage or database

◆ Sink

- Writes transformed data out
- Defines:
 - Target dataset
 - Write mode (Append/Overwrite/Upsert)
 - Partitioning options
- Supports Delta, Parquet, SQL, etc.

Use: Persist transformed results

ADF Components – Data Flow

◆ Transformations

These are the steps that modify or refine data.

Select

- Choose columns
- Rename them
- Drop unnecessary fields

Filter

- Remove rows based on conditions
Example: `age > 18`

Derived Column

- Create new columns using expressions
Example: `fullName = firstName + " " + lastName`

Aggregate

- Group data
Example: `SUM(sales)` by region, `COUNT` customers

Sort

- Order rows by one or more keys

Join

- Combine two streams based on keys
Types:
 - Inner, Left, Right, Full

Surrogate Key

- Generate sequential IDs
Use for dimensional modeling

Union

- Combine two streams vertically (stack)

Exists / Not Exists

- Check if rows exist in another dataset

ADF Components – Control Flow

In **Azure Data Factory (ADF)**, **Control Flow** is the orchestration layer of a pipeline.

It defines **how and when activities execute**, including sequencing, branching, looping, and error handling.

Control Flow allows to:

- Run activities **sequentially**
- Run activities **in parallel**
- Apply **conditions**
- Implement **loops**
- Handle **success/failure paths**
- Pass **parameters & variables**

Activity Name	Purpose	Common Use Case
Execute Pipeline	Calls another pipeline	Modular pipeline design
If Condition	Executes activities based on condition (True/False)	Conditional processing
Switch	Multi-branch logic (like case statement)	Dynamic routing based on value
ForEach	Loops through items (array)	Process multiple files/tables
Until	Repeats activities until condition is met	Polling / wait for external job
Wait	Pauses execution for defined time	Delay between steps
Web Activity	Calls REST API / external service	Trigger external system
Set Variable	Assign value to variable	Store temporary values
Append Variable	Add value to array variable	Collect loop results
Validation	Validate dataset before execution	Check file existence
Fail	Explicitly fail the pipeline	Custom error handling

ADF Components – Parameters & Variables

In **Azure Data Factory (ADF)**, **Parameters** and **Variables** are used to make pipelines dynamic and reusable. They help pass values and store temporary data during execution.

What are Parameters?

Parameters are **inputs passed to a pipeline** (or Data Flow, Dataset, Linked Service).

They are:

- Defined at design time
- Passed at runtime
- **Immutable** (cannot change during execution)

What are Variables?

Variables are used to **store temporary values during pipeline execution**.

They:

- Are defined inside pipeline
- Can change value
- Used in control flow logic

Feature	Parameters	Variables
Value changes during run?	No	Yes
Passed externally?	Yes	No
Scope	Pipeline-level or activity-level	Pipeline only
Used for	Dynamic input	Loop counters, flags

ADF Components - Integration Runtime

An **Integration Runtime** is the **compute infrastructure** used by Azure Data Factory to:

- Move data between systems
- Transform data
- Connect to on-premises and cloud sources
- Execute activities in a pipeline

Pipelines define *what to do*

Integration Runtime defines *how and where it runs*

Types of Integration Runtime

1. Azure Integration Runtime (Azure IR)

Runs in Azure-managed infrastructure

Used for:

- Cloud-to-cloud data movement
- Running Data Flows
- Most common use case

Examples:

- Azure SQL → ADLS
- Blob → Synapse

No infrastructure to manage.

2. Self-Hosted Integration Runtime (SHIR)

Runs on your own machine or server

Used for:

- On-premises databases
- Private network data sources
- Hybrid integration

Examples:

- On-prem SQL Server → Azure Blob
- File server → ADLS

Installed on a VM or on-prem server.

3. Azure SSIS Integration Runtime

Used to run SSIS packages in Azure

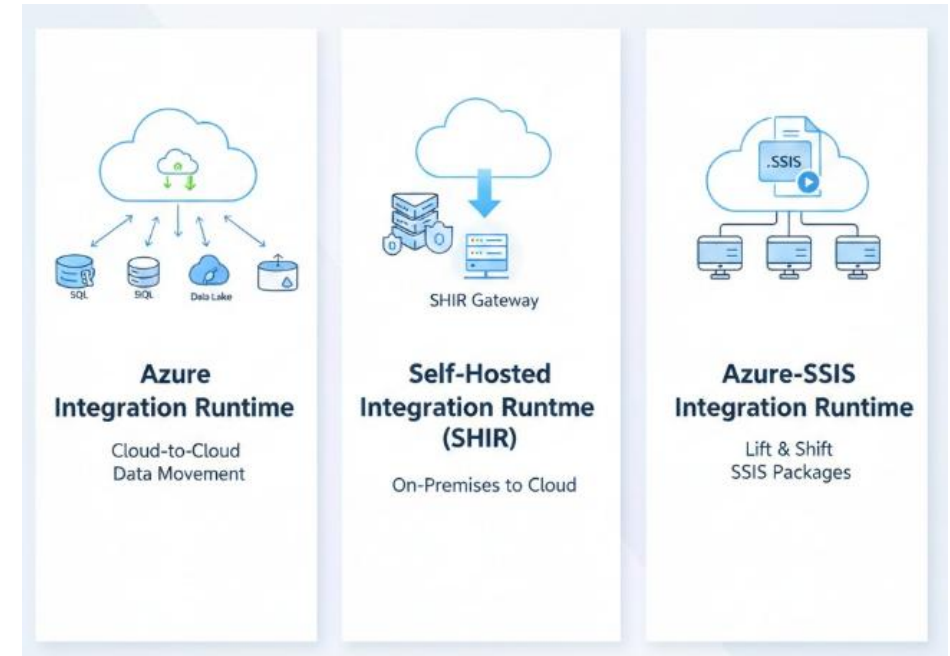
Used for:

- Lift-and-shift SSIS workloads
- Legacy ETL migration

Examples:

- Existing SSIS packages

Specialized use case.



Multi Cloud HealthCare ETL Data Engineering Project

Project Overview

This project is a **Multi-Cloud Data Engineering** solution designed to break down data silos in the healthcare sector.

By integrating fragmented data from AWS, GCP, and Azure, the platform establishes a **Unified Medallion Lakehouse** that provides a holistic view of patient journeys, hospital operations, and pharmaceutical trends.

Core Project Objectives

The primary goal is to synchronize disparate datasets into a single source of truth using **Azure Data Factory (ADF)** and **Databricks**, enabling the generation of high-value Hospital and Drug KPIs.

Multi Cloud HealthCare ETL Data Engineering Project

Tech Stack

- **Azure Data Factory (ADF)** - orchestration & ingestion
- **AWS S3, GCP BigQuery, Azure SQL** - data sources
- **Azure Data Lake Storage Gen2 (ADLS)** - storage
- **Azure Databricks (PySpark)** - transformation
- **Delta Lake** – Bronze / Silver / Gold architecture
- **Incremental Load + Audit + Reject handling + Metadata**

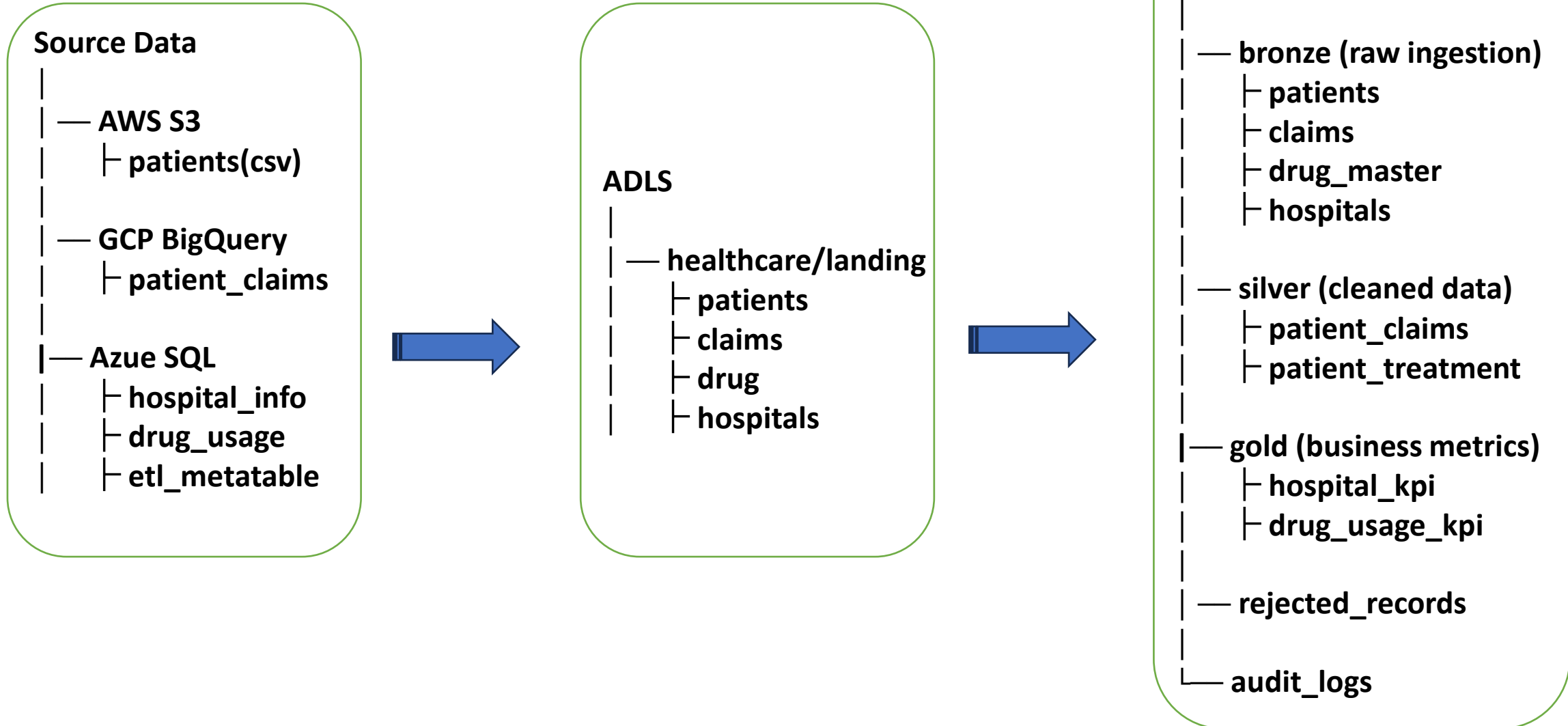
This is designed exactly like a **real production healthcare analytics pipeline**.

Source Systems

Source	System	Data
Patient Data	AWS S3	CSV files
Claims	GCP BigQuery	Claim transactions
Drug Master	Azure SQL	Drug reference
Hospital Info	Azure SQL	Hospital metadata

Multi Cloud HealthCare ETL Data Engineering Project

Data Flow Architecture



Multi Cloud HealthCare ETL Data Engineering Project

ADF Pipeline Design

Pipeline Name: pl_healthcare_multicloud_pipeline

Trigger (Daily)

Lookup Activity

(Get the last read timestamp from metadata table)



Copy Patient Data (AWS S3 → ADLS Landing)



Copy Claims Data (BigQuery → ADLS Landing)



Copy Drug Master (Azure SQL → ADLS Landing)



Copy Hospital Info (Azure SQL → ADLS Landing)

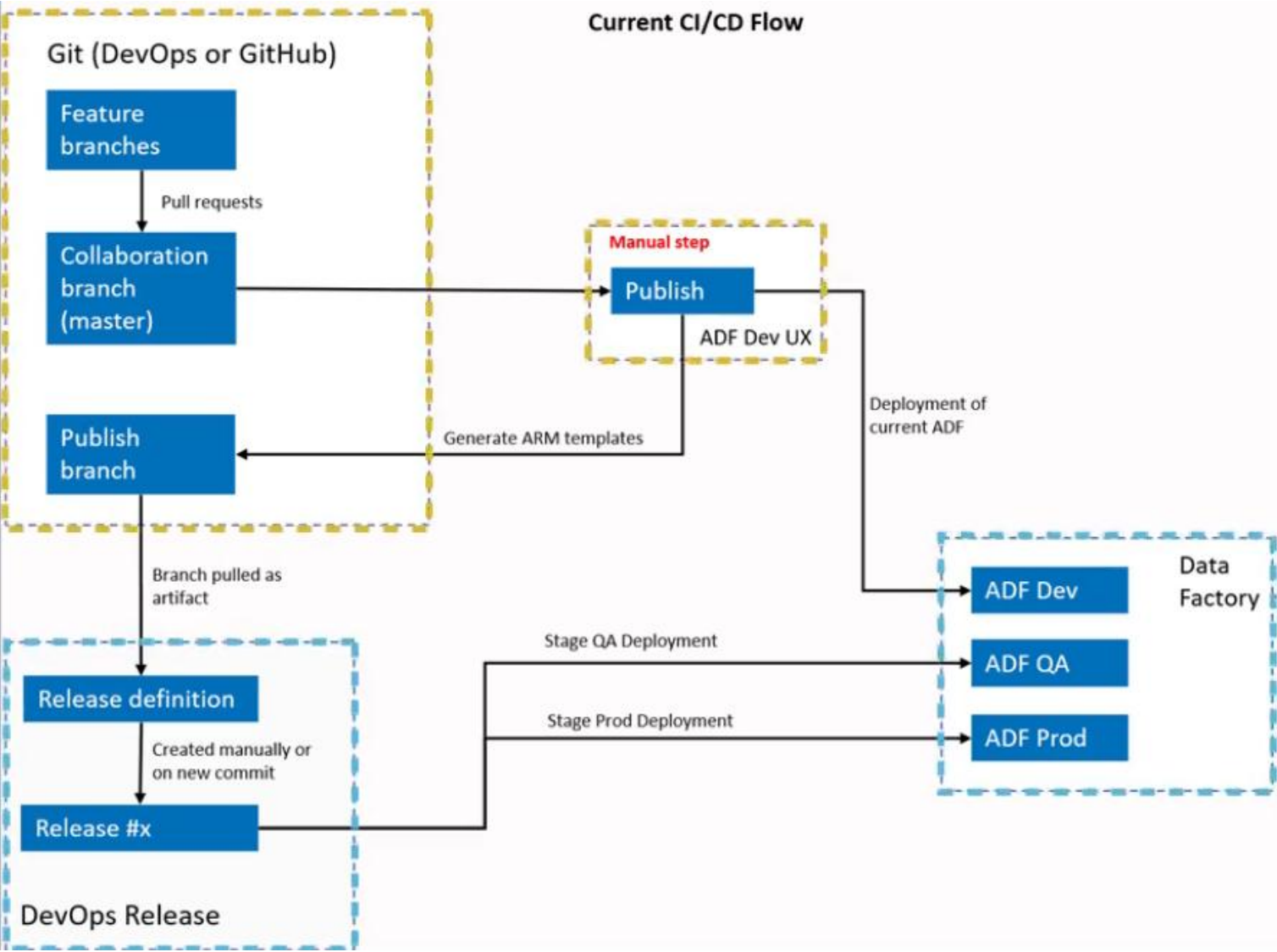


Databricks Notebook Activity(Process Data)



Script Activity(Update metadata Table)

Azure DevOps



Continuous Integration

Version Control: All data pipeline definitions, including datasets, linked services, and pipeline JSON, are stored in a version control system like Azure Repos or GitHub.

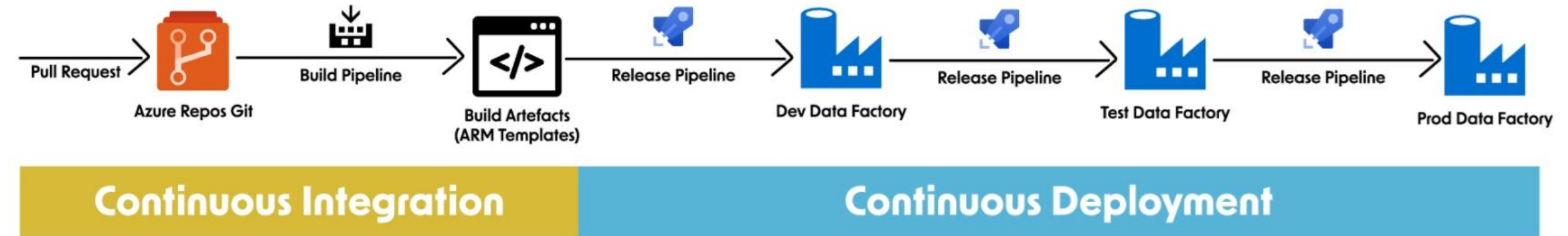
Automated Build: Upon committing changes to the repository, automated processes validate and integrate these changes with the existing codebase.

Continuous Deployment

Release: Changes that have passed through CI are prepared and packaged for release.

Deploy: These packages are automatically deployed to production and other specified environments, ensuring that the latest version is always operational.

Azure DevOps



A build pipeline generates artefacts such as ARM templates



A release pipeline deploys these artefacts to different environments



Blob Storage

LANDING

ORDER_ITEMS
ORDER_ID
LINE_ITEM_ID
PRODUCT_ID
UNIT_PRICE
QUANTITY

ORDERS
ORDER_ID
ORDER_DATE
CUSTOMER_ID
ORDER_STATUS
STORE_ID

STORES
STORE_ID
STORE_NAME
WEB_ADDRESS
LATITUDE
LONGITUDE

CUSTOMERS
CUSTOMER_ID
FULL_NAME
EMAIL_ADDRESS

PRODUCTS
PRODUCT_ID
PRODUCT_NAME
UNIT_PRICE



Data Lake Gen2

RAW

ORDER_ITEMS
ORDER_ID
LINE_ITEM_ID
PRODUCT_ID
UNIT_PRICE
QUANTITY

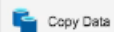
ORDERS
ORDER_ID
ORDER_DATE
CUSTOMER_ID
ORDER_STATUS
STORE_ID

STORES
STORE_ID
STORE_NAME
WEB_ADDRESS
LATITUDE
LONGITUDE

CUSTOMERS
CUSTOMER_ID
FULL_NAME
EMAIL_ADDRESS

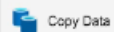
PRODUCTS
PRODUCT_ID
PRODUCT_NAME
UNIT_PRICE

Copy Data



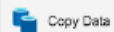
Copy Data

Copy Data



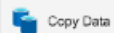
Copy Data

Copy Data



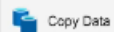
Copy Data

Copy Data



Copy Data

Copy Data



Copy Data



Data Lake Gen2

RAW

ORDERS

ORDER_ID
ORDER_DATETIME
CUSTOMER_ID
ORDER_STATUS
STORE_ID

ORDER_ITEMS

ORDER_ID
LINE_ITEM_ID
PRODUCT_ID
UNIT_PRICE
QUANTITY

STORES

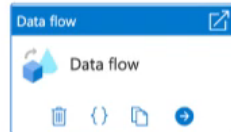
STORE_ID
STORE_NAME
WEB_ADDRESS
LATITUDE
LONGITUDE

PRODUCTS

PRODUCT_ID
PRODUCT_NAME
UNIT_PRICE

CUSTOMERS

CUSTOMER_ID
FULL_NAME
EMAIL_ADDRESS



CLEANSED

ORDERS

ORDER_ID *integer*
ORDER_DATE *date*
CUSTOMER_ID *integer*
ORDER_STATUS *string*
STORE_ID *integer*
UPDATED_TIMESTAMP *timestamp*

ORDER_ITEMS

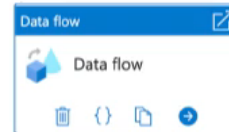
ORDER_ID *integer*
PRODUCT_ID *integer*
UNIT_PRICE *double*
QUANTITY *integer*
UPDATED_TIMESTAMP *timestamp*

STORES

STORE_ID *integer*
STORE_NAME *string*
LATITUDE *double*
LONGITUDE *double*
UPDATED_TIMESTAMP *timestamp*

PRODUCTS

PRODUCT_ID *integer*
PRODUCT_NAME *string*
UNIT_PRICE *double*
UPDATED_TIMESTAMP *timestamp*



STRUCTURED

ORDERS

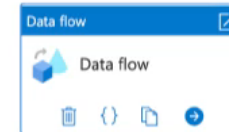
ORDER_ID *integer*
ORDER_DATE *date*
CUSTOMER_ID *integer*
ORDER_STATUS *string*
STORE_ID *integer*
PRODUCT_ID *integer*
UNIT_PRICE *double*
QUANTITY *integer*
SUBTOTAL *double*
UPDATED_DATE *timestamp*

STORES

STORE_ID *integer*
STORE_NAME *string*
LATITUDE *double*
LONGITUDE *double*
UPDATED_TIMESTAMP *timestamp*

PRODUCTS

PRODUCT_ID *integer*
PRODUCT_NAME *string*
UNIT_PRICE *double*
UPDATED_TIMESTAMP *timestamp*



ANALYTICS

STORES_ORDERS_MONTHLY

STORE_NAME
LATITUDE
LONGITUDE
ORDER_STATUS
MONTH_YEAR
TOTAL_AMOUNT

PRODUCTS_ORDERS_MONTHLY

PRODUCT_NAME
MONTH_YEAR
ORDER_STATUS
TOTAL_AMOUNT

Azure Databases

Azure Databases are cloud-based data storage services provided by **Microsoft**.

They help you **store, manage, secure, and scale data** without maintaining physical servers.

Type	Azure Database Name	Data Format	Query Method	Best For
Relational (SQL)	Azure SQL Database Azure SQL Managed Instance Azure Database for MySQL Azure Database for PostgreSQL	Tables (Rows & Columns)	SQL	Business apps, ERP, CRM
NoSQL	Azure Cosmos DB	JSON / Documents	API-based (SQL API, Mongo API, etc.)	Global apps, IoT, real-time systems
In-Memory (Cache)	Azure Cache for Redis	Key-Value	Fast Lookup	Caching, session storage
Analytics / Warehouse	Azure Synapse SQL Pool Azure Data Explorer	Structured / Columnar	SQL / KQL	Reporting, BI, big data analytics